

人工智能课程设计实验报告

机器人寻径避障问题



学 号 2351520

姓 名 毛星博

专 业 基础学科拔尖基地(计算机)

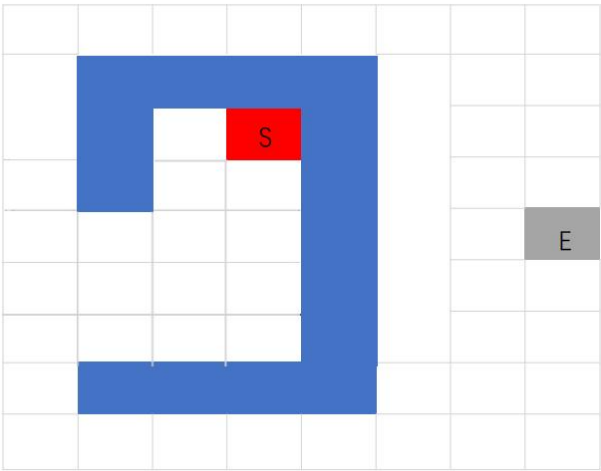
授课老师 武妍

一、问题概述

机器人避障寻径是机器人领域的一项重要技术，该技术要求实现机器人在复杂环境中的自主导航。在现实生活中，机器人的自主导航需要考虑各种因素，包括环境的复杂性、障碍物的位置和形状、机器人的运动能力等。如何解决这些问题是机器人避障寻径任务的研究难点。基于规则的传统路径规划方法，如 A* 算法、Dijkstra 算法等，已经在机器人路径规划中得到广泛应用，并能够在大部分场景中实现较高的准确度。这些算法的核心思想是通过搜索算法在地图中找到最优的路径。这些算法具有计算复杂度低、实现简单等优点。

1.1 机器人避障寻径问题的描述

在本实验中，给定如下的机器人避障寻径场景图：



其中 S 为机器人寻径的起点，E 为寻径的终点，蓝色区域为无法通过的障碍。要求机器人从 S 点出发，绕开障碍，到达 E 点。本实验需要参与者自行设计核心算法规则完成机器人从起点到终点的最短路径搜索。

1.2 问题的分析

该问题的核心挑战在于如何在考虑障碍物约束的同时，高效地搜索出一条从起点到终点的最优（最短）路径。环境的复杂性（障碍物的形状和分布）以及对路径最优性的要求，使得需要采用合适的搜索算法。

传统的基于规则的路径规划方法，如 A* 算法、Dijkstra 算法等，在机器人路径规划中得到了广泛应用。这些算法的核心思想是通过启发式搜索或广度优先搜索在地图中寻找最优路径。

一开始我采用的是广度搜索这类无启发式算法来实现，因为实现逻辑简单并且能够保证找到最优解。但是广搜算法的搜索效率比较低，在本实验的特定场景图中，广度搜索算法甚至会扩展场景中的每一个色块。

因此，为了提高算法效率，我改为选用 A* 算法来解决此问题，因为它在保证找到最优解的同时，通常比 Dijkstra 等无启发式算法具有更高的搜索效率。

1.3 本项目的实现目标

- (1) 实现基于 A* 算法的最优路径搜索逻辑。
- (2) 在给定的场景图中找到从起点到终点的最短路径。
- (3) (选做) 实现搜索过程的动态可视化展示。
- (4) (选做) 设计并测试更复杂的随机生成场景图。
- (5) (选做) 分析所设计算法的局限性, 及改进点

1.4 问题的求解思路

首先, 问题的核心是在网格中寻找最短路径。在一个给定的二维网格地图 (GRID) 中, 找到一条从起点 `START_POS` 到终点 `END_POS` 的路径, 这条路径必须避开所有标记为障碍物 (值为 1) 的格子, 并且是所有可行路径中总步数最少的那一条。

我本次采用的 A* 算法是启发式搜索算法, 换句话说它是一种“有方向感”的智能搜索。它试图优先探索那些“看起来”更有希望靠近终点的格子, 同时又不忽略实际已经走过的距离。这种“有方向感”是通过一个评估函数 $f(n) = g(n) + h(n)$ 来实现的。 $g(n)$: 从起点走到当前格子 n 实际已经花费的步数。这个值是精确的。 $h(n)$: 从当前格子 n 走到终点估计还需要多少步。这只是一个猜测 (启发值), 用于指导方向。 $f(n)$: 通过格子 n 到达终点的总估计步数。A* 算法每次都选择当前 $f(n)$ 值最小的格子进行下一步探索。

注: 在这个简单的二维场景图中, $g(n)$ 的值可以置为从起点出发, 探索到这一个点所花费的步数。这个值是准确的。初始值可以置为无穷大, `START_POS` 的 $g(n)$ 值为 0; 而 $h(n)$ 的值, 我通过计算当前点到终点的曼哈顿距离, 也就是横坐标和纵坐标差值的绝对值之和来得知。这不一定准确, 只是一个猜测, 毕竟中间可能会有障碍物的阻挡。但是障碍物的存在只会让实际路径更长, 因为需要绕路。它不会让实际路径比没有障碍物时的最短路径 (即曼哈顿距离) 更短。因此, 曼哈顿距离 $h(n)$ 总是小于等于从 n 到终点的实际最短路径成本。它满足可采纳性。

二、算法原理与设计

2.1 A*算法的设计

A*算法的功能是在给定的二维栅格地图中, 高效地搜索并找到一条从指定起点到指定终点的、避开所有障碍物的、总代价 (在本项目中指步数) 最小的路径。它通过结合实际已走代价 $g(n)$ 和到终点的启发式估计代价 $h(n)$ 来评估节点的优先级, 优先探索最有希望位于最短路径上的节点, 从而在保证找到最优解的前提下, 减少不必要的搜索范围。

2.1.1 数据结构:

开放列表 (Open Set): 这是 A* 算法的核心, 用于存放那些已经被发现但其邻居尚未完全检查的节点。为了能快速找到 $f(n)$ 最小的节点, 我使用了一个最小优先队列 (在 Python 中用 `heapq` 实现, 对应代码中的 `open_set_heap`), 它存储 (f_score , 节点坐标) 这样的元组,

并自动根据 `f_score` 排序。同时，为了能快速检查一个节点是否已经在开放列表中（避免重复添加或进行更新判断），我还使用了一个哈希集合（对应代码中的 `open_set_hash`），它只存储节点坐标，用于快速检查节点是否已经探索过。

代价记录 (`g_score`): (代码中使用字典 `g_score`) 来存储从起点到达每个已知节点的当前最小实际代价 `g(n)`。

父节点记录 (`parent`): 为了能在找到终点后回溯出完整路径，还需要一个数据结构（代码中使用字典 `parent`）来记录每个节点是从哪个父节点转移过来的。例如，`parent[B] = A` 表示我们是从节点 A 移动到节点 B 才得到了到达 B 的当前最短路径。

2.2.2 算法的执行流程:

初始化: 将起点加入开放列表 (`open_set_heap` 和 `open_set_hash`)，设置起点的 `g_score` 为 0，`parent` 为 `None`。

循环搜索: 只要开放列表不为空，就重复以下步骤:

从优先队列 `open_set_heap` 中取出具有最小 `f_score` 的节点，记为当前节点 `current`。

将 `current` 从 `open_set_hash` 中移除（表示该节点已被访问或正在处理其最优路径）。

检查 `current` 是否为终点。如果是，则搜索成功，跳转到步骤 3（路径回溯）。

如果不是终点，则考察 `current` 的所有有效邻居（在网格内、非障碍物）。

对于每个邻居 `neighbor`，计算通过 `current` 到达它的 `tentative_g_score`。

如果这条路径比之前记录的到达 `neighbor` 的路径更短 (`tentative_g_score < g_score[neighbor]`)，则更新 `neighbor` 的 `g_score` 和 `parent`（记录 `current` 为其父节点），并重新计算其 `f_score`。

如果 `neighbor` 不在 `open_set_hash` 中（即首次发现，或者之前的路径较差已被移除），则将其加入 `open_set_heap` 和 `open_set_hash`。

2.2.3 路径回溯得到最终路径:

一旦在上述循环中找到了终点，就需要利用之前记录的 `parent` 信息来构建实际的最短路径。

这个过程称为回溯:

从终点节点开始。

查找该节点的父节点（通过 `parent` 字典）。

将当前节点添加到路径列表的前端（或者先顺序添加再反转列表）。

将父节点设为新的当前节点。

重复这个过程，直到当前节点是起点（起点的父节点为 `None`）。

最终得到的节点序列（从起点到终点）就是找到的最短路径。在代码中，这个路径被存储在全局列表 `path` 中。

2.2 可视化算法的设计

可视化算法的功能是：利用图形界面直观地展示 A* 算法的搜索过程和最终找到的路径。这有助于理解算法的执行逻辑、调试代码，并清晰地呈现结果。

2.2.1 数据结构与关键元素:

Tkinter Canvas: 使用 Tkinter 库的 Canvas 组件作为绘图区域, 每个格子对应画布上的一个矩形区域。

颜色定义 (COLORS 字典): 定义不同状态 (如起点、终点、障碍物、已访问、待扩展、路径) 对应的颜色字符串。

格子尺寸 (CELL_SIZE): 定义每个格子在画布上绘制的像素大小。

更新函数:

update_cell(canvas, row, col, color): 负责在画布的指定行列位置绘制一个指定颜色的矩形 (格子)。

update_cell_with_text(canvas, row, col, color, text): 在绘制格子的基础上, 在其中心添加指定的文本 (如 'S', 'E', 或 g_score 值)。

draw_grid(canvas, grid, start_pos, end_pos): 负责绘制整个初始网格状态, 清除旧内容并根据 grid 数据绘制所有格子及其初始状态。

Tkinter Root Window (root): 主窗口对象, 用于管理事件循环和界面更新。

2.2.2 算法的流程:

初始化: 在 A* 算法开始前, 调用 draw_grid 绘制初始地图布局。

标记待扩展节点: 当 A* 算法将一个新节点 neighbor 加入开放列表时, 如果该节点不是终点, 则调用 update_cell 将其格子颜色更新为“待扩展”状态。

标记已访问节点: 当 A* 算法从开放列表取出节点 current 并开始处理时, 如果该节点不是起点或终点, 则调用 update_cell_with_text 将其格子颜色更新为“已访问”状态, 并显示其当前的 g_score。

标记最终路径: 当 A* 算法找到终点并执行路径回溯时, 对于回溯到的每个路径节点, 调用 update_cell_with_text 将其格子颜色更新为“路径”状态, 并显示其 g_score。

动画效果实现: 在每次调用 update_cell 或 update_cell_with_text 更新界面后, 刷新画布, 并引入短暂延迟, 从而创建出逐步搜索和路径构建的动画效果。

2.3 随机生成复杂场景图算法的设计

随机生成算法的功能是: 创建具有不同尺寸、不同起点/终点位置以及不同障碍物密度和分布的二维地图, 用于测试 A* 算法在更复杂场景下的性能。

2.3.1 数据结构:

rows, cols: 随机生成的地图行数和列数。

start_pos, end_pos: 随机生成的起点和终点坐标。

obstacle_density: 控制障碍物在地图中出现的概率 (例如 0.2 表示 20% 的格子是障碍物)。

Grid 表示: 仍然使用二维列表来存储生成的地图, 0 代表可通行, 1 代表障碍物。

2.3.2 算法的执行流程:

随机确定地图的 rows 和 cols。

随机确定起点 start_pos 和终点 end_pos 的坐标 (row, col), 确保它们在地图边界内, 并且起点不等于终点。

确定本次生成的障碍物密度 obstacle_density。

初始化网格: 创建一个 `rows x cols` 的二维列表, 初始时所有元素都设为 `0` (可通行)。

放置障碍物: 遍历网格的每一个格子 `(r, c)`: 生成一个 `0` 到 `1` 之间的随机浮点数。确保起点/终点可通行: 在放置完所有随机障碍物后, 强制将起点 `start_pos` 和终点 `end_pos` 对应的格子值设回 `0`, 确保它们一定是可通行的, 否则 `A*` 算法无法开始或结束。

返回地图: 将生成好的二维列表 `grid` 以及对应的 `rows, cols, start_pos, end_pos` 返回给主程序使用。

2.4 整体算法流程

1. 准备阶段:

生成地图尺寸、起点、终点和障碍物。

初始化优先队列 `open_set_heap`、辅助集合 `open_set_hash`、代价字典 `g_score`、父节点字典 `parent`。

绘制当前地图的初始状态。

2. 搜索阶段:

将起点加入开放列表 `open_set_heap` 和 `open_set_hash`。

循环直到找到终点或开放列表为空:

从 `open_set_heap` 取出 `f` 值最低的节点 `current`。

可视化: 将 `current` 标记为“已访问”。

判断: 如果 `current` 是终点, 则转到路径回溯阶段。

扩展: 检查 `current` 的有效邻居 `neighbor`。

更新: 如果通过 `current` 到达 `neighbor` 的路径更优, 则更新 `neighbor` 的 `g_score`、`parent` 和 `f_score`。

入队: 如果 `neighbor` 是新发现的 (不在 `open_set_hash`), 则加入 `open_set_heap` 和 `open_set_hash`, 并可视化标记为“待扩展”。

3. 路径回溯阶段:

从终点开始, 利用 `parent` 字典反向追踪父节点, 直到起点。

可视化: 将路径上的节点标记为“路径”颜色, 并显示 `g_score`。

将路径坐标存入 `path` 列表。

显示找到路径的消息。

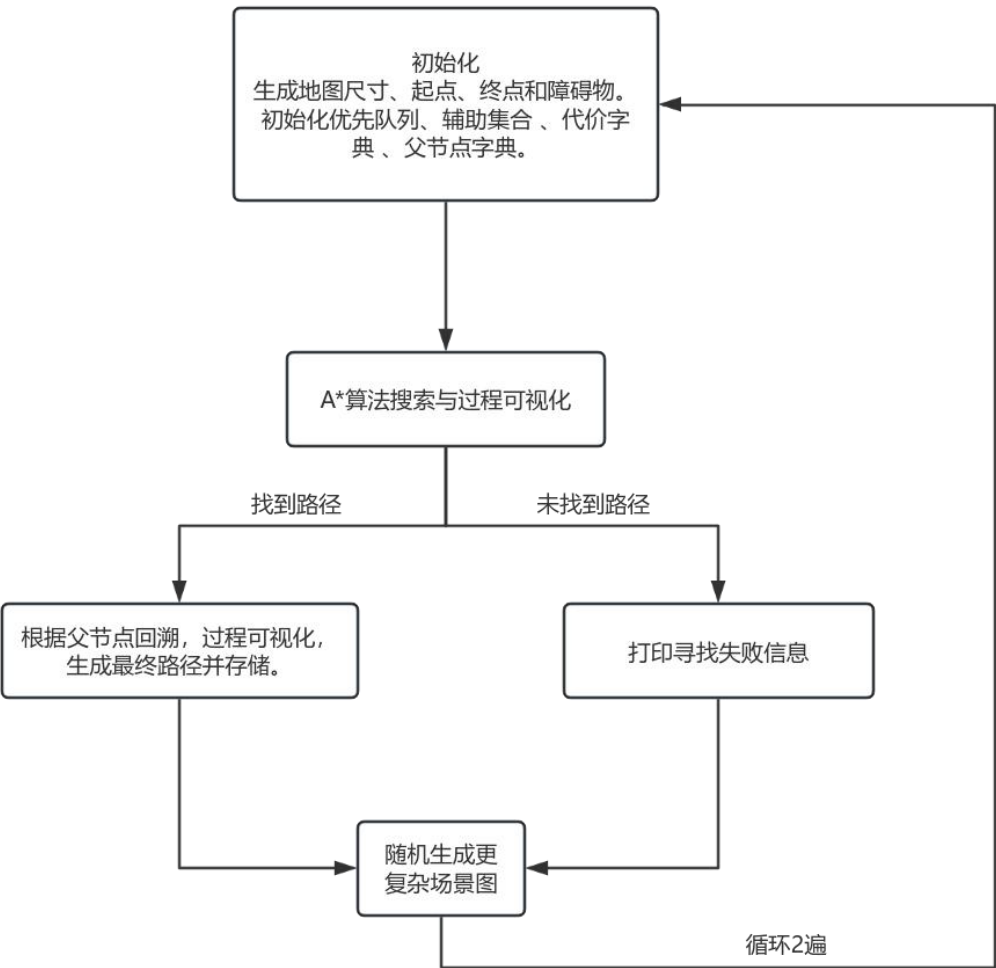
4. 结束阶段:

如果开放列表为空仍未找到终点, 显示未找到路径的消息。

`Tkinter` 窗口关闭后, 在控制台打印最终的 `path` 列表。

返回步骤 1 准备下一个复杂场景地图, 或结束程序。

2.5 算法流程图



三、算法的实现

本项目基于 Python 语言，利用 `heapq` 库实现优先队列，`tkinter` 库进行可视化，并结合字典和列表数据结构来实现 A* 算法。

3.1 数据结构定义：

GRID 表示栅格地图。每个元素代表一个格子，0 表示可通行，1 表示障碍物。在随机生成时，会动态创建类似结构的二维列表。

COLORS 存储了不同格子状态（可通行、障碍物、起点、终点、已访问、待扩展、路径）与 `Tkinter` 颜色字符串的映射，用于可视化。

open_set_heap 实现 A* 算法的开放列表（优先队列）。存储元组 `(f_score, (row, col))`。保证了列表始终维持最小堆结构，总是返回 `f_score` 最小的节点。

open_set_hash 存储当前有效的节点坐标。用于 `O(1)` 复杂度的查找，判断节点是否已在开放列表中，以及处理堆中可能存在的冗余节点。

`g_score` 键为节点坐标(row, col)，值为从起点到该节点的已知最短实际代价。
`parent` 键为子节点坐标(row, col)，值为其父节点坐标。用于在找到终点后回溯路径。
`path` 用于存储最终回溯得到的最短路径上的所有节点坐标序列。

```
# --- 默认参数 ---
DEFAULT_ROWS = 9
DEFAULT_COLS = 8
DEFAULT_START_POS = (2, 3)
DEFAULT_END_POS = (4, 7)

# 预定义网格（用于第一次运行）
GRID = [
    [0, 0, 0, 0, 0, 0, 0, 0], # Row 0
    [0, 1, 1, 1, 1, 0, 0, 0], # Row 1
    [0, 1, 0, 0, 1, 0, 0, 0], # Row 2 (S at 2,3)
    [0, 1, 0, 0, 1, 0, 0, 0], # Row 3
    [0, 0, 0, 0, 1, 0, 0, 0], # Row 4 (E at 4,7)
    [0, 0, 0, 0, 1, 0, 0, 0], # Row 5
    [0, 0, 0, 0, 1, 0, 0, 0], # Row 6
    [0, 1, 1, 1, 1, 0, 0, 0], # Row 7
    [0, 0, 0, 0, 0, 0, 0, 0], # Row 8
]

# 颜色定义
COLORS = {
    "free": "white",      # 可通行区域
    "obstacle": "blue",   # 障碍物
    "start": "red",       # 起点
    "end": "yellow",      # 终点
    "visited": "orange",  # 已访问节点
    "queued": "lightgray", # 待访问节点
    "path": "green",      # 最短路径
}

# Tkinter 可视化参数
CELL_SIZE = 30
DELAY_MS = 50          # 动画延迟
TEXT_COLOR = "black"    # 文本颜色
path = []              # 储存最终路径

open_set_heap = [(0, start)] # 优先队列，存储待扩展节点
open_set_hash = {start} # 集合，存储已扩展节点
parent = {start: None} # 字典，存储每个节点的父节点
g_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) } # 字典，存储每个节点到起点的距离
g_score[start] = 0 # 起点到起点的距离为 0
```

```
f_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) } # 字典, 存储每个节点到终点的距离
f_score[start] = heuristic(start, end) # 起点到终点的距离
```

3.2 核心函数: astar(grid_data, start, end, canvas, root_window)

实现 A* 搜索算法的主逻辑, 并调用可视化函数。

初始化上述数据结构

```
open_set_heap = [(0, start)] # 优先队列, 存储待扩展节点
open_set_hash = {start} # 集合, 存储已扩展节点
parent = {start: None} # 字典, 存储每个节点的父节点
g_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) } # 字典, 存储每个节点到起点的距离
g_score[start] = 0 # 起点到起点的距离为 0
f_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) } # 字典, 存储每个节点到终点的距离
f_score[start] = heuristic(start, end) # 起点到终点的距离
```

搜索

```
while open_set_heap:
    current_f, current_pos = heapq.heappop(open_set_heap) # 从优先队列中取出 f 值最小的节点
    if current_pos not in open_set_hash:
        continue # 如果当前节点不在已扩展节点集合中, 则跳过
    open_set_hash.remove(current_pos) # 从已扩展节点集合中移除当前节点

    current_row, current_col = current_pos
    # 已访问节点涂色并显示与起点的距离
    if current_pos != start and current_pos != end:
        update_cell_with_text(canvas, current_row, current_col, COLORS['visited'],
g_score[current_pos])

    root_window.update_idletasks()
    root_window.after(DELAY_MS)

    # 扩展
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        new_row, new_col = current_row + dr, current_col + dc # 计算邻居节点的坐标
        neighbor = (new_row, new_col)
        if 0 <= new_row < rows and 0 <= new_col < cols and grid_data[new_row][new_col] == 0:
            tentative_g_score = g_score[current_pos] + 1 # 计算邻居节点到起点的距离
            if tentative_g_score < g_score[neighbor]:
                parent[neighbor] = current_pos # 更新邻居节点的父节点为当前节点
                g_score[neighbor] = tentative_g_score # 更新邻居节点到起点的距离
                f_score[neighbor] = tentative_g_score + heuristic(neighbor, end) # 计算邻居节点到终点的距离

            if neighbor not in open_set_hash: # 如果邻居节点不在待扩展节点集合中
```

```

        heapq.heappush(open_set_heap, (f_score[neighbor], neighbor)) # 将邻居节点
加入优先队列

        open_set_hash.add(neighbor) # 将邻居节点加入已扩展节点集合
        # 待扩展节点涂色
        if neighbor != end:
            update_cell(canvas, new_row, new_col, COLORS['queued'])
            root_window.update_idletasks()
            root_window.after(DELAY_MS // 2)

```

若到达，则回溯

```

        # 找到终点，执行回溯
        if current_pos == end:
            path_found = True
            path_len = 0
            temp_current = end

            while temp_current is not None: # 从终点开始回溯
                path.insert(0, temp_current) # 将当前节点插入路径列表的开头
                parent_node = parent.get(temp_current) # 获取当前节点的父节点
                if parent_node is not None: # 如果父节点存在
                    path_len += 1 # 路径长度加 1
                    if temp_current != end and temp_current != start: # 如果当前节点不是终点或起点
                        g_val = g_score.get(temp_current, '?') # 获取当前节点到起点的距离
                        update_cell_with_text(canvas, temp_current[0], temp_current[1],
COLORS['path'], g_val) # 更新当前节点的颜色为路径颜色，并显示距离
                        root_window.update_idletasks()
                        root_window.after(DELAY_MS)

                    temp_current = parent_node # 更新当前节点为父节点
            messagebox.showinfo("完成", f"已找到最优路径！长度为: {path_len}")
            break

```

循环结束后若 path_found 为 False，则显示未找到路径

```

if not path_found:
    print("未找到路径!")
    messagebox.showinfo("结果", "无法从起点到达终点。")

```

3.3 模块输入输出:

主要输入:

grid_data: 当前要搜索的地图。

start: 起点坐标 (row, col)。

end: 终点坐标 (row, col)。

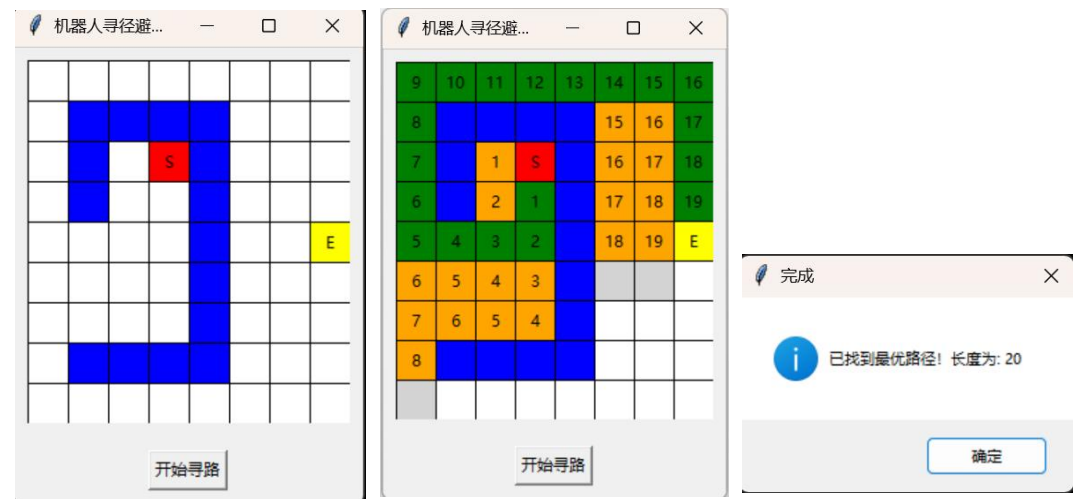
主要输出:

可视化结果: 动态展示搜索过程和最终路径。

打印全局列表 path 中存储的最终路径坐标序列。

四、实验结果

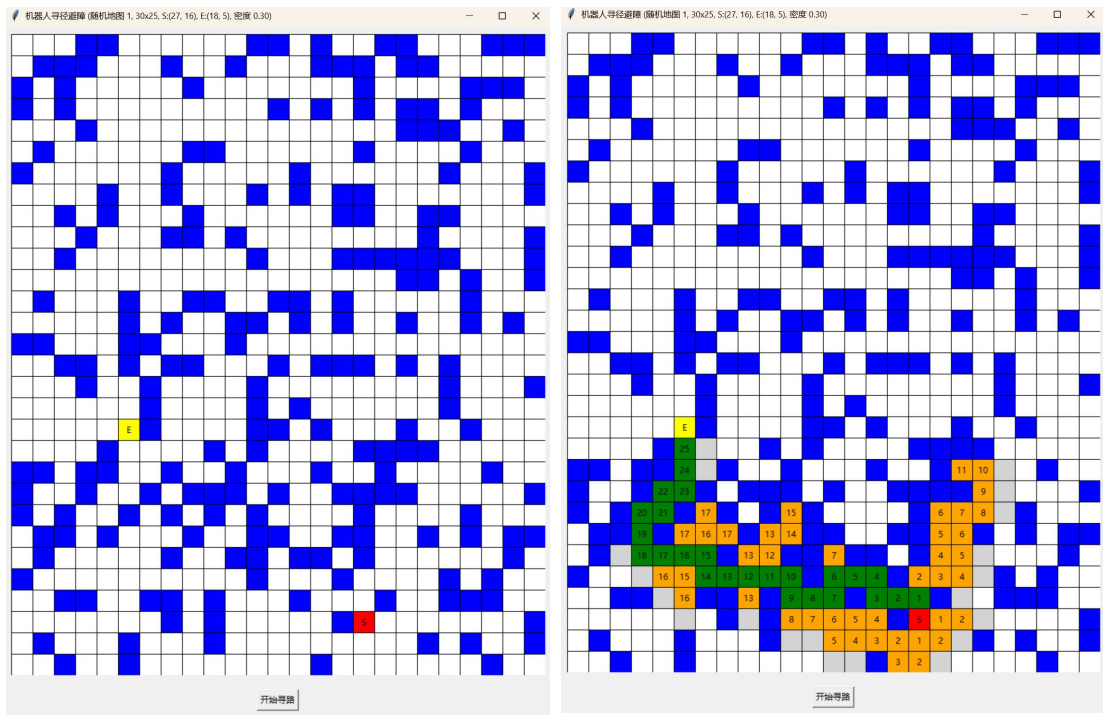
初始给定场景
搜索过程与结果可视化：



最终路径：

path: [(2, 3), (3, 3), (4, 3), (4, 2), (4, 1), (4, 0), (3, 0), (2, 0), (1, 0), (0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (0, 5), (0, 6), (0, 7), (1, 7), (2, 7), (3, 7), (4, 7)]

随机生成复杂场景
可视化：



最终路径:

```
path: [(27, 16), (26, 16), (26, 15), (26, 14), (25, 14), (25, 13), (25, 12), (26, 12), (26, 11), (26, 10), (25, 10), (25, 9), (25, 8), (25, 7), (25, 6), (24, 6), (24, 5), (24, 4), (24, 3), (23, 3), (22, 3), (22, 4), (21, 4), (21, 5), (20, 5), (19, 5), (18, 5)]
```

五、总结与分析

5.1 本次实验遇到的问题和解决方法

Open Set 管理与冗余处理:

最初在理解 A* 算法时, 对于如何高效管理开放列表以及处理节点可能被多次以不同 f_score 加入优先队列的情况有些困惑。

解决方案: 通过结合使用 heapq 优先队列和 set 集合 (open_set_hash), 解决了这个问题。优先队列保证了总是能取出 f_score 最低的节点, 而集合用于快速判断节点是否应被处理 (if current_pos not in open_set_hash: continue), 有效避免了对旧的、非最优路径节点的重复处理。

Tkinter 可视化:

在实现动态可视化时, 发现界面更新不及时或动画效果不明显。

解决方案: 使用了 root_window.update_idletasks() 来强制刷新界面, 以及 root_window.after() 来引入适当的延迟, 从而实现了平滑、可观察的逐步动画效果。同时, 调整 DELAY_MS 的值以在观察清晰度与执行速度间取得平衡。

5.2 对算法的理解与局限性分析

通过本次实验, 我对 A* 算法有了更深入的理解:

A* 是一种启发式搜索算法, 其核心在于评估函数 $f(n) = g(n) + h(n)$, 它智能地平衡了已走路径的实际代价和到目标的估计代价。启发函数 $h(n)$ 的选择至关重要, 其“可采纳性”(不高于实际代价)是保证 A* 找到最优解的关键。曼哈顿距离在本项目的四向移动网格中是可采纳且计算简单的选择。优先队列 (Open Set) 是算法效率的关键, 它确保了每次都能探索最有希望的节点。

局限性分析:

静态环境: A* 算法本身适用于静态环境, 对于环境中出现动态变化的障碍物无法直接有效处理。

内存消耗: 对于非常大的地图, 存储 Open Set、g_score 和 parent 字典可能会占用大量内存。
启发函数依赖: 算法效率很大程度上依赖于启发函数的质量。

5.3 算法的改进点

动态环境适应: 采用 D* Lite 等增量式搜索算法, 当环境发生变化时, 可以快速更新路径而无需完全重新规划。

内存优化: 对于大规模地图, 可以考虑 JPS 等内存占用更低的算法。

非统一代价处理: 修改 `tentative_g_score` 的计算方式, 使其能反映不同移动的实际代价 (例如, 对角线移动代价设为 `sqrt(2)`, 或者根据地形赋予不同格子的穿越代价)。

5.4 实验的收获与思考

本次实验使我受益匪浅:

将课堂上学到的 A* 算法理论知识, 通过 Python 代码一步步实现出来, 加深了对算法每个步骤、每个数据结构作用的理解。

可视化: 实现动态可视化不仅让结果展示更直观, 更重要的是在调试过程中极大地帮助了理解算法的执行流程, 快速定位问题。

熟练了 `heapq`, `tkinter`, `random` 等 Python 库的使用, 并实践了函数封装、数据结构选择、Lambda 参数传递等编程技巧。

遇到了从算法逻辑到可视化同步等一系列问题, 通过分析、查阅资料和调试, 最终找到了解决方案, 锻炼了独立解决问题的能力不仅实现了算法, 还分析了其局限性和改进方向, 认识到没有完美的算法, 需要根据具体应用场景选择和优化。

总的来说, 这次实验不仅巩固了对 A* 算法的掌握, 也提升了将算法应用于实际问题并进行可视化的能力, 同时引发了我对更复杂环境和更优算法的进一步思考。

六、附件-源代码

```
import tkinter as tk
from tkinter import messagebox
import heapq # 用于优先队列
import time
import math
import random

# --- 默认参数 ---
DEFAULT_ROWS = 9
DEFAULT_COLS = 8
DEFAULT_START_POS = (2, 3)
DEFAULT_END_POS = (4, 7)
# 预定义网格 (用于第一次运行)
GRID = [
    [0, 0, 0, 0, 0, 0, 0, 0], # Row 0
    [0, 1, 1, 1, 1, 0, 0, 0], # Row 1
    [0, 1, 0, 0, 1, 0, 0, 0], # Row 2 (S at 2,3)
    [0, 1, 0, 0, 1, 0, 0, 0], # Row 3
    [0, 0, 0, 0, 1, 0, 0, 0], # Row 4 (E at 4,7)
    [0, 0, 0, 0, 1, 0, 0, 0], # Row 5
```

```

    [0, 0, 0, 0, 1, 0, 0, 0], # Row 6
    [0, 1, 1, 1, 1, 0, 0, 0], # Row 7
    [0, 0, 0, 0, 0, 0, 0, 0], # Row 8
]

# 颜色定义
COLORS = {
    "free": "white",      # 可通行区域
    "obstacle": "blue",   # 障碍物
    "start": "red",       # 起点
    "end": "yellow",      # 终点
    "visited": "orange",  # 已访问节点
    "queued": "lightgray", # 待访问节点
    "path": "green",      # 最短路径
}

# Tkinter 可视化参数
CELL_SIZE = 30
DELAY_MS = 50          # 动画延迟
TEXT_COLOR = "black"   # 文本颜色
path = []              # 储存最终路径

# --- 函数：生成随机网格 ---
def generate_random_grid(rows, cols, obstacle_density=0.3, start=DEFAULT_START_POS,
end=DEFAULT_END_POS):
    """生成一个具有随机障碍物的网格（不保证起点终点可达或有解）"""
    grid = [[0 for _ in range(cols)] for _ in range(rows)] # 初始化全为 0
    for r in range(rows):
        for c in range(cols):
            # 根据密度随机放置障碍物
            if random.random() < obstacle_density and (r, c) != start and (r, c) != end:
                grid[r][c] = 1 # 1 表示障碍物
    return grid

# --- 辅助函数 ---
def update_cell(canvas, row, col, color):
    """更新画布上指定格子的颜色"""
    x1 = col * CELL_SIZE
    y1 = row * CELL_SIZE
    x2 = x1 + CELL_SIZE
    y2 = y1 + CELL_SIZE
    canvas.delete(f"cell_{row}_{col}")
    canvas.create_rectangle(x1, y1, x2, y2, fill=color, outline="black",
tags=(f"cell_{row}_{col}", "cell"))

def update_cell_with_text(canvas, row, col, color, text):
    """更新画布上指定格子的颜色并在中心绘制文本"""
    update_cell(canvas, row, col, color)
    x_center = col * CELL_SIZE + CELL_SIZE / 2

```

```

        y_center = row * CELL_SIZE + CELL_SIZE / 2
        canvas.delete(f"text_{row}_{col}")
        canvas.create_text(x_center, y_center, text=str(text), fill=TEXT_COLOR,
tags=(f"text_{row}_{col}", "text"))
def draw_grid(canvas, grid, start_pos, end_pos):
    """绘制初始网格状态"""
    canvas.delete("cell")
    canvas.delete("text")
    rows, cols = len(grid), len(grid[0])
    for r in range(rows):
        for c in range(cols):
            pos = (r, c)
            if pos == start_pos:
                update_cell_with_text(canvas, r, c, COLORS['start'], "S")
            elif pos == end_pos:
                update_cell_with_text(canvas, r, c, COLORS['end'], "E")
            else:
                update_cell(canvas, r, c, COLORS['free'] if grid[r][c] == 0 else
COLORS['obstacle'])
# --- A* 算法函数 ---
def heuristic(pos1, pos2):
    """计算曼哈顿距离作为启发式函数 H"""
    r1, c1 = pos1
    r2, c2 = pos2
    return abs(r1 - r2) + abs(c1 - c2)
# A* 算法
def astar(grid_data, start, end, canvas, root_window):
    """执行 A* 搜索并可视化过程"""
    global path
    path.clear()
    rows, cols = len(grid_data), len(grid_data[0])
    open_set_heap = [(0, start)] # 优先队列, 存储待扩展节点
    open_set_hash = {start} # 集合, 存储已扩展节点
    parent = {start: None} # 字典, 存储每个节点的父节点
    g_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) } # 字典, 存储每个节点到起点的距离
    g_score[start] = 0 # 起点到起点的距离为 0
    f_score = { (r, c): float('inf') for r in range(rows) for c in range(cols) } # 字典, 存储每个节点到终点的距离
    f_score[start] = heuristic(start, end) # 起点到终点的距离
    path_found = False

    while open_set_heap:

```

```

        current_f, current_pos = heapq.heappop(open_set_heap) # 从优先队列中取出 f 值最小的节点
    if current_pos not in open_set_hash:
        continue # 如果当前节点不在已扩展节点集合中，则跳过
    open_set_hash.remove(current_pos) # 从已扩展节点集合中移除当前节点
    current_row, current_col = current_pos
    # 已访问节点涂色并显示与起点的距离
    if current_pos != start and current_pos != end:
        update_cell_with_text(canvas, current_row, current_col, COLORS['visited'], g_score[current_pos])
        root_window.update_idletasks()
        root_window.after(DELAY_MS)
    # 找到终点，执行回溯
    if current_pos == end:
        path_found = True
        path_len = 0
        temp_current = end
        while temp_current is not None: # 从终点开始回溯
            path.insert(0, temp_current) # 将当前节点插入路径列表的开头
            parent_node = parent.get(temp_current) # 获取当前节点的父节点
            if parent_node is not None: # 如果父节点存在
                path_len += 1 # 路径长度加 1
                if temp_current != end and temp_current != start: # 如果当前节点不是终点或起点
                    g_val = g_score.get(temp_current, '?') # 获取当前节点到起点的距离
                    update_cell_with_text(canvas, temp_current[0], temp_current[1], COLORS['path'], g_val) # 更新当前节点的颜色为路径颜色，并显示距离
                    root_window.update_idletasks()
                    root_window.after(DELAY_MS)
                temp_current = parent_node # 更新当前节点为父节点
            messagebox.showinfo("完成", f"已找到最优路径！长度为: {path_len}")
            break
    # 扩展
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        new_row, new_col = current_row + dr, current_col + dc # 计算邻居节点的坐标
        neighbor = (new_row, new_col)
        if 0 <= new_row < rows and 0 <= new_col < cols and grid_data[new_row][new_col] == 0:
            tentative_g_score = g_score[current_pos] + 1 # 计算邻居节点到起点的距离
            if tentative_g_score < g_score[neighbor]:
                parent[neighbor] = current_pos # 更新邻居节点的父节点为当前节点
                g_score[neighbor] = tentative_g_score # 更新邻居节点到起点的距离

```

```

        f_score[neighbor] = tentative_g_score + heuristic(neighbor, end) #
计算邻居节点到终点的距离
        if neighbor not in open_set_hash: # 如果邻居节点不在待扩展节点集合中
            heapq.heappush(open_set_heap, (f_score[neighbor], neighbor)) #
将邻居节点加入优先队列
        open_set_hash.add(neighbor) # 将邻居节点加入已扩展节点集合
        # 待扩展节点涂色
        if neighbor != end:
            update_cell(canvas, new_row, new_col, COLORS['queued'])
            root_window.update_idletasks()
            root_window.after(DELAY_MS // 2)

    if not path_found:
        print("未找到路径!")
        messagebox.showinfo("结果", "无法从起点到达终点。")
# --- 主程序入口 ---
if __name__ == "__main__":
    for i in range(3):
        current_rows = DEFAULT_ROWS
        current_cols = DEFAULT_COLS
        current_start_pos = DEFAULT_START_POS
        current_end_pos = DEFAULT_END_POS
        current_grid = GRID
        title_suffix = " (预定义地图)"
        if i > 0: # 第二次和第三次循环使用随机参数
            # 随机确定尺寸
            current_rows = random.randint(20, 30)
            current_cols = random.randint(20, 30)
            # 随机确定起点和终点, 确保不重复且在界内
            while True:
                r_start = random.randint(0, current_rows - 1)
                c_start = random.randint(0, current_cols - 1)
                current_start_pos = (r_start, c_start)
                r_end = random.randint(0, current_rows - 1)
                c_end = random.randint(0, current_cols - 1)
                current_end_pos = (r_end, c_end)
                if heuristic(current_start_pos, current_end_pos) > 10:
                    break
            # 生成随机网格
            density = 0.3
            current_grid = generate_random_grid(current_rows, current_cols, density,
current_start_pos, current_end_pos)
            # 强制确保起点和终点是可通行的 (值为 0)
            current_grid[current_start_pos[0]][current_start_pos[1]] = 0
            current_grid[current_end_pos[0]][current_end_pos[1]] = 0

```

```

        title_suffix = f" (随机地图 {i}, {current_rows}x{current_cols},
S:{current_start_pos}, E:{current_end_pos}, 密度 {density:.2f})"

    # --- 创建 Tkinter 窗口和组件 ---
    root = tk.Tk()
    root.title("机器人寻径避障" + title_suffix)
    # 根据当前尺寸计算画布大小
    canvas_width = current_cols * CELL_SIZE
    canvas_height = current_rows * CELL_SIZE
    canvas = tk.Canvas(root, width=canvas_width, height=canvas_height,
borderwidth=0, highlightthickness=0)
    canvas.pack(pady=10, padx=10)
    # 绘制当前网格
    draw_grid(canvas, current_grid, current_start_pos, current_end_pos)
    # 创建启动按钮, 使用 lambda 的默认参数捕获当前迭代的参数
    start_button = tk.Button(root, text="开始寻路", command=lambda
grid=current_grid, start=current_start_pos, end=current_end_pos, r=root, c=canvas:
astar(grid, start, end, c, r))
    start_button.pack(pady=10)
    # 启动 Tkinter 主事件循环
    root.mainloop()
    # 在 Tkinter 窗口关闭后打印路径
    print("path:", path)

```